

EX. NO. : 11	CREATE JSX-SYNTAX BASED APPLICATION USING JSX
DATE :	

AIM:

To Create JSX-syntax based application using JSX.

ALGORITHM:

1. Start the React app and initialize state variable count = 0.
2. Render UI → display a heading, a paragraph, and a button showing Clicked 0 times.
3. Wait for user action → if the button is clicked, trigger the onClick event.
4. Update state → increase count by 1 (count = count + 1).
5. Re-render UI → show the updated button text "Clicked {count} times".

PROGRAM:

```

<!DOCTYPE html>

<html lang="en">

  <head>

    <meta charset="UTF-8" />

    <title>JSX Example</title>

    <script crossorigin src="https://unpkg.com/react-dom@18/umd/react-dom.development.js"></script>

  </head> <body>

    <div id="root"></div>

    <script type="text/babel">

function App() {

  const [count, setCount] = React.useState(0);

return (

  <div>

    <h1>JSX Syntax Example</h1>

```

```
<p>Hello, this text is written in <strong>JSX</strong>!</p>

  <button onClick={() => setCount(count + 1)}>

    Clicked {count} times

  </button>

</div>

);

}

const root = ReactDOM.createRoot(document.getElementById("root"));

</script>

</body>

</html>
```

OUTPUT:

JSX Syntax Example

Hello, this text is written in **JSX**!

Clicked 3 times

Class Performance	
Record	
Viva	
Total	
Signature	

RESULT:

Thus, the create JSX-syntax based application using JSX was displayed.

EX. NO. : 12	HANDLE USER EVENTS IN REACT COMPONENTS
DATE :	

AIM:

To handle user events in react components.

ALGORITHM:

1. Initialize component with two states: text (for input) and message (for button click feedback).
2. Render UI with a heading, button, message area, input box, and typed text display.
3. Handle button click using handleClick() → update message to “Button was clicked!”.
4. Handle input change using handleChange() → update text with whatever the user types.
5. Update output dynamically → React re-renders to show the latest message and text values.

PROGRAM:

```

<!DOCTYPE html>

<html>

  <head>

    <meta charset="UTF-8" />

    <title>User Events in React</title>

    <script src="https://unpkg.com/@babel/standalone/babel.min.js"></script>

  </head> <body>

    function App() {

      const [text, setText] = React.useState("");

      const [message, setMessage] = React.useState("");

      const handleClick = () => {

        setMessage("Button was clicked!"); }

      const handleChange = (event) => {

```

```

    setText(event.target.value);  };
return (

    <div style={{ fontFamily: "Arial", padding: "20px" }}>

        <h1>React User Events</h1>

        <button onClick={handleClick}>Click Me</button>

        <p>{message}</p>

        <input type="text" placeholder="Type something..." value={text} />

        <p>You typed: {text}</p>

    </div> ); }

ReactDOM.createRoot(document.getElementById("root")).render(<App />);

</script>

</body>

</html>

```

OUTPUT:

React User Events

Click Me

Button was clicked!

I'm Naveen in CSE B

You typed: I'm Naveen in CSE B

Class Performance	
Record	
Viva	
Total	
Signature	

RESULT:

Thus, the handle user events in react components was executed.

EX. NO. : 13	IMPLEMENT CONDITIONAL RENDERING IN A REACT APP
DATE :	

AIM:

To implement conditional rendering in a react app.

ALGORITHM:

1. Start the React app and initialize a state variable isLoggedIn as false.
2. Render the heading and a section that changes based on the value of isLoggedIn.
3. If isLoggedIn is true, display a welcome message with a Logout button.
4. If isLoggedIn is false, display a login message with a Login button.
5. Update the state when buttons are clicked and re-render the UI accordingly.

PROGRAM:

```

<!DOCTYPE html>

<html>

<head>

<meta charset="UTF-8" />

<title>Conditional Rendering in React</title>

<script src="https://unpkg.com/react@18/umd/react.development.js"></script>

</head>

<body>

  function App() {

    const [isLoggedIn, setIsLoggedIn] = React.useState(false);

    const handleLogin = () => setIsLoggedIn(true);

    const handleLogout = () => setIsLoggedIn(false);

    return (

      <div style={{ fontFamily: "Arial", padding: "20px" }}>

        <h1>Conditional Rendering Example</h1>

```

```
    { /* Conditional Rendering */ }
```

```
    { isLoggedIn ? (
```

```
      <div>
```

```
        <h2>Welcome, User! </h2>
```

```
        <button onClick={handleLogout}>Logout</button> </div>
```

```
    ) : ( <div>
```

```
      <h2>Please log in to continue.</h2>
```

```
      <button onClick={handleLogin}>Login</button>
```

```
    </div> ) }
```

```
  </div> ); }
```

```
ReactDOM.createRoot(document.getElementById("root")).render(<App />);
```

```
</script>
```

```
</body>
```

```
</html>
```

OUTPUT:

Conditional Rendering Example

Welcome, User!

Logout

Class Performance	
Record	
Viva	
Total	
Signature	

RESULT:

Thus, the conditional rendering in react was executed.

EX. NO. : 14	RENDER LISTS WITH UNIQUE KEYS FROM DATA
DATE :	

AIM:

To render lists with unique keys from data in react.

ALGORITHM:

1. Initialize a list of objects (users) with unique id values.
2. Use the map() function to loop through the list.
3. For each item, create a React element ().
4. Assign the key attribute using the unique id of each item.
5. Render the list inside a parent container ().

PROGRAM:

```
<!DOCTYPE html>

<html>

<head>

  <meta charset="UTF-8" />

  <title>Render Lists in React</title>

  <script src="https://unpkg.com/react@18/umd/react.development.js"></script>

</head> <body>

  function App() {

    const users = [

      { id: 1, name: "Naveen", age: 22 },

      { id: 2, name: "Sunil", age: 25 },

      { id: 3, name: "Selva", age: 30 } ];

    return (

      <div style={{ fontFamily: "Arial", padding: "20px" }}>

        <h1>Rendering Lists in React</h1>
```

```

    <ul>

    {users.map((user) => (

    <li key={user.id}> {user.name} — Age: {user.age}

    </li>

    )) } </ul>

</div>

); }

const root = ReactDOM.createRoot(document.getElementById("root"));

root.render(<App />);

</script>

</body>

</html>

```

OUTPUT:

Rendering Lists in React

- Naveen — Age: 22
- Sunil — Age: 25
- Selva — Age: 30

Class Performance	
Record	
Viva	
Total	
Signature	

RESULT:

Thus, the render lists with unique keys from data in react was executed.

EX. NO. : 15	MANAGE COMPONENT STATE AND LIFECYCLE USING HOOKS
DATE :	

AIM:

To manage component state and lifecycle using hooks.

ALGORITHM:

1. Initialize state variables using useState (e.g., count, time).
2. Display state values in the UI.
3. Update state when user interacts (e.g., button click changes count).
4. Use useEffect to run lifecycle logic (e.g., start timer on mount).
5. Clean up resources in useEffect return function when component unmounts.

PROGRAM:

```

<!DOCTYPE html>

<html>

  <head>

    <meta charset="UTF-8" />

    <title>React Hooks Example</title>

    <script src="https://unpkg.com/react-dom@18/umd/react-dom.development.js"></script>

  </head> <body>

    function App() {

      const [count, setCount] = React.useState(0);

      const [time, setTime] = React.useState(new Date().toLocaleTimeString());

      React.useEffect(() => {

        console.log("Component mounted ;

        const timer = setInterval(() => {

          setTime(new Date().toLocaleTimeString()); }, 1000);

        return () => { clearInterval(timer);

```

```
console.log("Component unmounted"); }; }, []);

return (

  <h1>React Hooks: State & Lifecycle</h1>

  <p>Counter: {count}</p>

  <button onClick={() => setCount(count + 1)}>Increment</button>

  <button onClick={() => setCount(count - 1)}>Decrement</button>

  <h2>Current Time: {time} </h2> ); }

const root = ReactDOM.createRoot(document.getElementById("root"));

root.render(<App />);

</script>

</body>

</html>
```

OUTPUT:

React Hooks: State & Lifecycle

Counter: 2

Increment

Decrement

Current Time: 10:48:48 PM

Class Performance	
Record	
Viva	
Total	
Signature	

RESULT:

Thus, the manage component state and lifecycle using hooks was executed.

EX. NO. : 16	INSTALLATION AND SETUP OF NODE.JS, NPM AND EXPRESS FRAMEWORK
DATE :	

AIM :

To install node.js and npm using command prompt and setup the project folder in visual studio.

DEFINITION :

NODE.JS

Node.js is an open-source and cross-platform JavaScript runtime environment. It is a popular tool for almost any kind of project. Node.js runs the V8 JavaScript engine, the core of Google Chrome, outside of the browser. This allows Node.js to be very performant. Node.js allows you to write JavaScript on both client and server.

NPM

NPM (Node Package Manager) is the default package manager for Node.js, used to install, share, and manage code packages (libraries or modules) for JavaScript projects. It allows developers to easily include reusable code from a vast ecosystem of open-source packages hosted on the npm registry. With NPM, you can manage project dependencies, run scripts, and control project versions using a simple command-line interface.

EXPRESS.JS

Express.js is a fast, minimal, and flexible web application framework for Node.js that simplifies the process of building server-side applications and APIs. It provides robust features such as routing, middleware support, and HTTP utility methods, allowing developers to handle requests, responses, and application logic efficiently. Express is widely used for building RESTful APIs and web applications, making it a core part of the MEAN and MERN stacks. Its simplicity and scalability make it ideal for both small projects and large-scale applications.

STEPS TO INSTALL :

STEP-1

Check if already installed:

Open a terminal and run:

node -v

npm -v

```
C:\Windows\System32\cr x + v
Microsoft Windows [Version 10.0.26100.4484]
(c) Microsoft Corporation. All rights reserved.

C:\MERN LAB>node -v
v22.16.0

C:\MERN LAB>npm -v
11.4.1

C:\MERN LAB>|
```

If not installed:

Download & Install from: <https://nodejs.org>

STEP 2:

Create a Project Folder Use your terminal or command prompt:

```
C:\Windows\System32\cr x + v
Microsoft Windows [Version 10.0.26100.4484]
(c) Microsoft Corporation. All rights reserved.

C:\MERN LAB>mkdir session-example

C:\MERN LAB>cd session-example

C:\MERN LAB\session-example>
```

STEP 3:

Initialize Node.js Project `npm init -y` This creates a package.json file.

```
C:\MERN LAB\session-example>npm init -y
Wrote to C:\MERN LAB\session-example\package.json:

{
  "name": "session-example",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs"
}
```

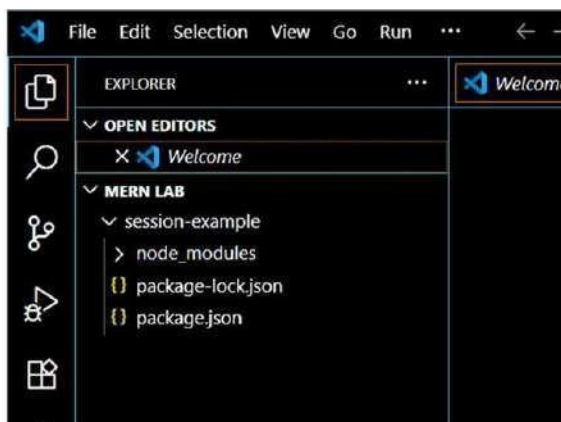
STEP 4:

Install Required npm Packages Run the following command to install: npm install express express-session

```
C:\MERN LAB\session-example>npm install
up to date, audited 1 package in 501ms
found 0 vulnerabilities
C:\MERN LAB\session-example>npm install express
added 67 packages, and audited 68 packages in 3s
14 packages are looking for funding
  run 'npm fund' for details
found 0 vulnerabilities
C:\MERN LAB\session-example>npm install express-session
added 7 packages, and audited 75 packages in 2s
14 packages are looking for funding
  run 'npm fund' for details
found 0 vulnerabilities
C:\MERN LAB\session-example>
```

STEP 5:

Directory Overview Your folder should now contain:



Class Performance	
Record	
Viva	
Total	
Signature	

RESULT :

Thus, the installation of node.js, npm and express.js was successfully completed.

RADIKAA R
714024104167

EX. NO. : 17	BASIC EXPRESS APP SETUP
DATE :	

AIM :

To develop a basic web server using Express.js that simulates a hospital booking homepage with multiple navigation routes. This includes creating routes for the homepage, appointments, doctors, and contact sections. The experiment aims to demonstrate how Express handles simple routing and responses on a local server.

ALGORITHM :

- Step 1: Import the Express module using require.
- Step 2: Create an Express app using express().
- Step 3: Create a route for the homepage (/).
- Step 4: Create a route for the appointments page (/appointments).
- Step 5: Create a route for the doctors page (/doctors).
- Step 6: Create a route for the contact page (/contact).
- Step 7: Set a port number (e.g., 3000).
- Step 8: Start the server and print a message to show it is running.

PROGRAM:

```
import React from 'react';

import { BrowserRouter as Router, Routes, Route, Link } from 'react-router-dom';

function Home() {

  return <h2><img alt="Hospital icon" data-bbox="225 680 240 695"/> Welcome to Hospital Home Page</h2>;

}

function Appointments() {

  return <h2><img alt="Calendar icon" data-bbox="215 765 230 780"/> Book Your Appointment</h2>;

}

function Doctors() {

  return <h2><img alt="Stethoscope icon" data-bbox="235 855 250 870"/> List of Doctors</h2>;

}
```

```

function Contact() {
  return <h2>! Contact Us</h2>;
}

function App() {
  return (
    <Router>
    <div style={{ textAlign: 'center', marginTop: '2rem' }}>
      <h1>Hospital Booking System</h1>
      <nav style={{ marginBottom: '1.5rem' }}>
        <Link to="/" style={{ margin: '1rem' }}>Home</Link>
        <Link to="/appointments" style={{ margin: '1rem' }}>Appointments</Link>
        <Link to="/doctors" style={{ margin: '1rem' }}>Doctors</Link>
        <Link to="/contact" style={{ margin: '1rem' }}>Contact</Link>
      </nav>
    <Routes>
      <Route path="/" element={<Home />} />
      <Route path="/appointments" element={<Appointments />} />
      <Route path="/doctors" element={<Doctors />} />
      <Route path="/contact" element={<Contact />} />
    </Routes>
    </div>
  </Router>
  );
}

export default App;

```

OUTPUT:

1.Home Page:



2.Appointment Page:



Hospital Booking System

[Home](#)[Appointments](#)[Doctors](#)[Contact](#)

Book Your Appointment

Class Performance	
Record	
Viva	
Total	
Signature	

RESULT:

A basic hospital homepage using Express.js was successfully implemented. The server handles different routes effectively, providing appropriate responses for the homepage,

RADIKAA R
714024104167

appointments, doctors, and contact sections.